VIC: Object Detection in Baggage using Classic CV

Fotios Kapotos

February 13, 2026

Pipeline Description and Design Choices

Overview. We use a classical two-stage detector: handcrafted proposal generation, handcrafted feature extraction with supervised classification, then score-based filtering. The system is tuned on validation data and then reused for final inference.

Tuning Protocol. Execution has two phases. First, we cache proposal-level outputs on validation images and run a grid search over objectness threshold, class threshold, final-score threshold, and maximum boxes per image. This avoids recomputing proposals and features for every setting. Second, we load the best thresholds and run standard inference on validation and test data.

The code is available on github : https://github.com/fotisk07/VIC-Project

1 Region Proposal Strategy

Proposals are generated by combining three methods: Canny edges with morphological closing, adaptive thresholding with opening, and MSER region detection. We filter candidates using normalized area bounds, then merge duplicates with NMS. NMS scores are proportional to box area, and proposal caps limit runtime.

2 Feature Representation

Each proposal is described with geometry (aspect ratio, normalized area, center), contour statistics (extent, solidity, circularity, Hu moments), grayscale patch statistics, HSV histograms, LBP-like texture bins, gradient histograms, and compact HOG descriptors. Feature dictionaries are vectorized with `DictVectorizer`, which keeps feature engineering modular.

3 Training Sample Matching and Label Assignment

For each training image, every proposal is matched to the GT box with highest IoU. Label assignment follows fixed IoU rules:

- IoU ≥ 0.30: positive sample with the matched GT class.
- IoU ≤ 0.10: background sample.
- $0.10 <$ IoU < 0.30: ignored sample.

This procedure defines both objectness and class supervision.

4 Two-Stage Probabilistic Classification

We train a Logistic Regression objectness model and a class model (XGBoost in the reported setup). At inference, proposal confidence is

$$s = p_{\text{object}} \cdot p_{\text{class}}.$$

This decomposition stabilizes learning under class imbalance and lets the class model focus on positive regions.

Background proposals are heavily overrepresented, so we downsample negatives and enforce

$$N_{\text{bg}} \leq 2.5 \cdot N_{\text{pos}}.$$

Without this step, training is biased toward trivial background decisions.

5 Threshold Optimization

We cache objectness, class confidence, and final score for validation proposals, then run a grid search over threshold combinations and max boxes per image. This decouples expensive feature extraction from threshold selection and directly optimizes the evaluation score. The best configuration is saved and reused for final runs.

6 Empirical Performance

Validation score on 900 frames:

$$\textbf{Score} = 0.2602.$$

Per-class performance is summarized below:

Class	mIoU	Recall	Precision
Hammer	0.244	0.381	0.335
Knife	0.131	0.193	0.296
Gun	0.192	0.263	0.625
Wrench	0.159	0.214	0.653
HandCuffs	0.548	0.761	0.818
Bullet	0.154	0.194	0.253

Performance is strongest for structurally distinctive objects (notably handcuffs). Lower recall for knives and bullets is consistent with proposal fragmentation on thin objects.

Limitations

The main limitation is proposal quality: missed proposals cannot be recovered downstream. The model also has no box regression stage, so localization remains coarse. Finally, handcrafted features restrict representation capacity compared with learned deep features.